

기술 해설 문서

축구 영상 선수 기여도 분석 프로그램 해설

영상에서 좌표를 뽑는 5개 파일이 각각 무엇을 하는지,
그 좌표에서 기여도(SC · PR · PA · DPI · TC)가 어떻게 계산되는지,
그리고 두 단계 사이에서 지금 무엇이 끊어져 있는지.

대상 코드 · `app.py` · `bootstrap.py` · `tracker.py` · `draw_boxes.py` · `recover_summary.py`
`soccer_dpi_pa_v7.py` (기여도 계산)

작성일 · 2026-09-30

문서의 모든 수치는 코드에 적힌 실측값이거나, 코드를 직접 실행해 확인한 값이다.

목 차

이 문서의 구성

1. 전체 지도 — 프로그램은 두 단계로 나뉜다

1.1 두 단계 · 1.2 파일별 역할 · 1.3 데이터가 흘러가는 길

2. 측정 단계 (1) — 준비와 조작

2.1 bootstrap.py · 2.2 app.py · 2.3 설정이 결과를 바꾸는 방식

3. 측정 단계 (2) — tracker.py 가 하는 모든 일

3.1 검출 · 3.2 추적 · 3.3 사람 거르기 · 3.4 팀 나누기 · 3.5 골키퍼
3.6 공 (6단계) · 3.7 출력 CSV

4. 측정 단계 (3) — 검증과 복구 도구

4.1 draw_boxes.py · 4.2 recover_summary.py

5. 해석 단계 — 기여도는 이렇게 계산된다

5.1 무엇을 재려는 것인가 · 5.2 계산 전 준비 · 5.3 공 소유 판정
5.4 SC · 5.5 PR · 5.6 PA · 5.7 DPI와 TC · 5.8 진단 출력

6. 두 단계 사이 — 지금 끊어져 있는 다섯 곳

6.1 픽셀과 미터 · 6.2 열 이름 · 6.3 팀 번호 · 6.4 공 행 · 6.5 보간 표시

7. 사례 해부 — 지난번 결과표가 망가진 이유

8. 고치는 순서

9. 용어 사전

부록 A. CSV 열 전체 표 / 부록 B. 주요 상수 표

이 문서를 읽는 가장 빠른 길

시간이 없다면 **1장(전체 지도)** → **5장(기여도 계산)** → **7장(사례 해부)** 세 장만 읽어도 된다. 3장은 tracker.py 를 손볼 때 찾아보는 참고서에 가깝고, 6장과 8장은 지금 당장 무엇을 해야 하는지에 대한 답이다.

1 장

전체 지도 — 프로그램은 두 단계로 나뉜다

1.1 측정과 해석, 두 단계

이 프로그램은 성격이 완전히 다른 두 단계가 이어 붙은 구조다. 둘은 **파일 하나**로만 연결된다.

1단계 · 측정 (Measurement)

영상 → 좌표

"이 프레임의 3번 선수는 화면 어디에 있는가"를 답한다.
컴퓨터 비전 문제다. YOLO 검출, ByteTrack 추적, 색 군집, 오검출 필터.

해당 파일 : bootstrap.py, app.py, **tracker.py**,
draw_boxes.py, recover_summary.py

산출물 : results/<영상이름>/tracks.csv

2단계 · 해석 (Interpretation)

좌표 → 기여도

"그 위치에 서 있던 것이 팀에 얼마나 도움이 되었는가"를
답한다. 축구 전술과 통계 문제다. 공간 분할, 협력게임 배분,
순위 정규화.

해당 파일 : soccer_dpi_pa_v7.py

산출물 : sc_pr_result.xlsx (선수별 SC · PR · PA · DPI · TC)

가장 먼저 알아야 할 사실

올려주신 5개 파일 안에는 기여도를 계산하는 코드가 **한 줄도 없다**. 다섯 파일 전체를 기여 , DPI , Shapley , xlsx
로 검색해도 아무것도 나오지 않는다. 이 파일들은 전부 1단계(측정)다. 기여도 수식은 별도 파일인
soccer_dpi_pa_v7.py 에 있다. 두 단계가 같은 프로그램처럼 보이지만, 실제로는 **CSV 파일 하나를 사이에 두고
서로를 전혀 모르는 두 프로그램**이다.

이 구조 자체는 나쁘지 않다. 오히려 옳다. 측정과 해석을 섞어 놓으면 결과가 이상할 때 검출이 틀린 것인지 수식이
틀린 것인지 영원히 알 수 없기 때문이다. 다만 두 프로그램이 서로를 모른다는 것은, **둘을 잇는 규약(파일 형식·단위·
좌표계)을 사람이 직접 맞춰 줘야 한다**는 뜻이다. 지금 그 규약이 어긋나 있고, 그것이 6장의 내용이다.

1.2 파일별 역할 한눈에

파일	단계	하는 일	없으면 생기는 일
bootstrap.py 146행	준비	최초 1회 환경 설치. venv 생성 → PyTorch(GPU 감지해 CUDA/CPU 자동 선택) → Ultralytics → lap → OpenVINO → tkinterdnd2 순서로 설치하고, 끝나면 venv 파이썬으로 app.py 를 띄우고 물러난다.	설치 순서가 어긋나 추적 ID 가 전부 -1 이 되거나, CPU 버전 torch 가 GPU 를 덮어쓴다.
app.py 622행	조작	tkinter GUI. 영상 대기열, 드래그앤드롭, videos 폴더 감시, 프리셋(모델·해상도·프레임간격), 설정 저장 (config.json), 진행률·남은시간 표시, 로그. 실제 분석은 별도 스레드에서 <code>tracker.run_batch()</code> 를 부른다.	터미널에서 tracker.py 를 직접 돌려야 한다 (가능은 하다).
tracker.py 2,593행 — 핵심	측정	YOLO 검출 + ByteTrack 추적 + 경기장 밖 제거 + 그래픽 제거 + 팀 색 군집 + 골키퍼 판정 + 공 오검출 6단계 필터. 프레임마다 CSV 한 줄씩 적고 summary.json 을 남긴다.	좌표가 없다. 이 프로그램의 전부다.
draw_boxes.py 402행	검증	이미 뽑은 tracks.csv 를 원본 영상에 다시 그려 확인용 영상을 만든다. YOLO 를 다시 돌리지 않아 빠르다. 구간 자르기, 해상도 축소, 라벨 방식 선택 가능.	숫자가 맞는지 눈으로 확인할 방법이 사라진다. 매우 중요하다(4.1 참고).
recover_summary.py 116행	복구	tracks.csv 는 멀쩡한데 summary.json 만 없는 폴더를 복구한다. CSV 를 흘려 읽어 통계를 다시 계산해 summary.json 을 만든다.	'이미 한 건 건너뛰기'가 작동하지 않아 몇 시간짜리 분석을 처음부터 다시 돌게 된다.
soccer_dpi_pa_v7.py 1,957행 — 별도	해석	좌표 CSV/엑셀을 읽어 SC · PR · PA 를 계산하고 DPI · TC 로 합쳐 선수별 표를 만든다. 자체 검증 12항목과 진단 출력을 함께 낸다.	좌표만 있고 기여도가 없다.

1.3 데이터가 흘러가는 길

영상 파일 (mp4 · 1080p 45분 등)

videos/ 폴더에 넣거나 창에 끌어다 놓는다

app.py — 설정을 정하고 tracker 를 부른다

모델 / 해상도(imgsz) / 신뢰도(conf) / 프레임간격(vid_stride) / 각종 필터 스위치

tracker.py — 프레임을 하나씩 본다

YOLO 검출 → ByteTrack 으로 번호 유지 → 잔디 밖 사람 버림 → 상체 색으로 팀 판정 → 공 후보 검사 → CSV 에 한 줄 기록
분석이 끝난 뒤 : 팀 다수결 확정 · 골키퍼 트랙 확정 · 정지 그래픽 제거 · 공 궤적 잇기 · 공 보관

results/<영상이름>/tracks.csv

frame, time_sec, track_id, class_id, class_name, conf, x1,y1,x2,y2, cx,cy, **fx,fy**, w,h, team, interpolated — 단위는
화면 픽셀

⚠ 여기가 지금 끊어져 있다 (6장)

픽셀 → 미터 변환(호모그래피)이 어디에도 없고, 열 이름과 팀 번호 규약도 서로 맞지 않는다

soccer_dpi_pa_v7.py — 기여도 계산

공 소유 판정 → SC(공간 통제) · PR(압박) · PA(패스 유인) → DPI · TC · TC_adj

sc_pr_result.xlsx — 선수별 결과표

선수별 / 프레임별 / 프레임메타 / PA프레임별 / 검출된패스 / 표본검사 시트

2 장

측정 단계 (1) — 준비와 조작

2.1 bootstrap.py — 설치 순서가 곧 결과 품질이다

이 파일은 한 번만 도는 설치 스크립트다. 그런데 여기 적힌 **순서**가 나중의 데이터 품질을 직접 결정한다. 평범한 설치 스크립트가 아니라는 뜻이다. 중요한 대목이 세 곳 있다.

(1) torch 를 ultralytics 보다 먼저 설치한다

```
gpu = has_nvidia_gpu()                # nvidia-smi 가 도는지로 판단
if gpu: pip install torch torchvision --index-url .../whl/cu124
else:   pip install torch torchvision   # CPU 판
pip install ultralytics                # ← 반드시 torch 다음
```

ultralytics 를 먼저 깔면 의존성 해결 과정에서 CPU 전용 torch 휠을 끌어온다. 그러면 GPU 가 있는 PC 인데도 CPU 로 돌아가고, 45분 영상 분석이 몇 분에서 몇 시간으로 늘어난다. 코드에 "torch 를 먼저 깔아줘야 ultralytics 가 제멋대로 CPU 휠을 끌어오지 않는다" 라고 적혀 있는 이유다.

(2) lap 을 미리 설치한다 — 안 하면 첫 분석의 track_id 가 전부 -1 이 된다

```
pip install lap    # ByteTrack 의 헝가리안 매칭에 필요
```

이것이 왜 치명적인가

lap 이 없으면 ultralytics 가 첫 추적 도중에 알아서 설치를 시도하는데, 그 회차는 추적이 작동하지 않아 track_id 가 전부 -1 로 나온다. track_id 가 -1 이면 "같은 선수"라는 개념이 사라지고, 기여도 계산은 선수별 집계 자체가 불가능해진다. 몇 시간을 돌리고도 쓸 수 없는 CSV 가 나온다. 그래서 설치 시점에 미리 넣는다.

(3) 실패해도 되는 것과 아닌 것을 구분한다

OpenVINO(CPU 가속)와 tkinterdnd2(드래그앤드롭)는 설치에 실패해도 그냥 넘어간다. 없어도 프로그램이 도는 것들이다. 반면 torch · ultralytics · lap 은 실패하면 venv 를 통째로 지우고 중단한다. 반쯤 설치된 환경을 남기지 않기 위해서다.

2.2 app.py — 사람이 만지는 부분

GUI 코드지만 분석 품질과 직결되는 판단이 몇 가지 들어 있다.

프리셋 — 모델·해상도·프레임간격을 한 번에

프리셋	모델	해상도	프레임간격	성격
빠르게 (GPU 없을 때)	축구 전용	640	3	3프레임에 1장. GPU 가 없으면 자동으로 이것이 선택된다
보통	축구 전용	960	2	640 대비 약 1.8배 느림
정확하게 (GPU 필요)	축구 전용	1280	1	전 프레임. CPU 로는 매우 오래 걸린다

프리셋에 담긴 실측 근거

코드 주석에 같은 720p 영상으로 썸 값이 적혀 있다.

- COCO yolo11n @640 : 프레임당 **12.1명** 검출 (평균 신뢰도 0.21)
- 축구 전용 모델 @640 : 프레임당 **23.1명** 검출 (평균 신뢰도 0.82)

같은 해상도에서 두 배 차이다. 더 중요한 것은 **축구 모델은 해상도를 올려도 검출이 거의 안 는다**는 점이다 (640에서 23.1명 → 1600에서 25.0명). 해상도를 올리는 것은 COCO 모델의 약점을 돈으로 때우던 것이지 원래 필요한 일이 아니었다. 그래서 세 프리셋 모두 축구 전용 모델을 쓴다.

폴더 감시 — 복사 중인 파일을 건드리지 않는 방법

videos/ 폴더를 2초마다 확인하는데, 파일이 보이자마자 잡지 않는다. 파일 크기를 기억해 두었다가 **직전 확인 때와 크기가 같을 때만** 대기열에 넣는다. 복사가 끝나지 않은 파일을 분석하다 깨지는 것을 막는 장치다.

결과 영상 저장을 자동으로 끄지 않는다

예전에는 프레임간격이 2 이상이면 결과 영상 저장을 자동으로 끄다고 주석에 적혀 있다. 영상이 끊겨 보인다는 이유였다. 그런데 GPU 가 없는 PC 는 '빠르게'(간격 3)가 자동 적용되므로, 결국 **눈으로 확인할 수단이 말없이 사라지는** 결과가 됐다. 지금은 끄지 않는다. 주석의 표현이 정확하다 — "끊기는 영상이 없는 영상보다 낫다."

2.3 설정이 결과를 바꾸는 방식

설정	기본값	올리면 / 내리면
<code>imgsz</code> 해상도	1280	올리면 작은 물체(특히 공)를 잘 찾지만 느려진다. 축구 모델에서는 선수 검출에는 거의 영향이 없고 공 에만 크게 영향을 준다.
<code>conf</code> 신뢰도	0.10	내리면 검출이 늘지만 오검출도 는다. 이 프로그램은 낮게 잡고 뒤에서 물리 규칙으로 거르는 전략을 쓴다.
<code>vid_stride</code>	1	3으로 올리면 3배 빨라지지만 추적이 끊기기 쉬워진다 . 0.125초 간격이면 달리는 선수가 자기 몸 너비만큼 움직여 박스 겹침이 0 이 되기 때문이다. 이것이 7장에서 다룰 ID 파편화의 주원인이다.
<code>match_thresh</code>	0.9	ByteTrack 이 트랙을 이어붙일 때 허용하는 (1-IoU) 상한. 0.8→0.9 로 올렸을 때 실측: 추적ID 233→139개, 트랙 지속 7→13장, 검출 15.8→17.9명.
<code>track_buffer</code>	30	가려진 선수를 몇 프레임까지 기억할지. 크게 잡으면 ID 가 덜 끊기지만 엉뚱한 사람에게 옮겨붙을 여지가 생긴다.
<code>team_imgsz</code>	0(=imgsz)	팀 색을 뽑을 때만 쓰는 해상도. 640 에서는 상체가 20px 남짓이라 실측상 41%가 team=-1 로 뺐다 . 팀 색 보정은 영상 전체에서 16장만 보므로, 그 16장만 고해상도로 처리하면 비용은 거의 안 늘고 팀 구분만 좋아진다.

3 장

측정 단계 (2) — tracker.py 가 하는 모든 일

2,593행짜리 이 파일이 프로그램의 심장이다. 구조는 단순하다 — 프레임 하나를 받아 CSV 여러 줄을 뱉는 루프, 그리고 루프가 끝난 뒤 전체를 한 번 더 훑는 후처리. 어려운 것은 그 사이에 낀 필터들인데, 필터 하나하나가 "무엇을 잘못 잡았는가"라는 관찰에서 출발한다.

3.1 검출 — 무엇을 사람으로, 무엇을 공으로 보는가

`ModelProfile` 클래스(29행)가 "지금 쓰는 모델이 무엇을 아는지"를 정리한다. 두 종류를 지원한다.

모델 종류	클래스	따라오는 결과
COCO 일반 모델 <code>yolo11n/s/m/l/x</code>	<code>person(0)</code> , <code>sports ball(32)</code> 둘뿐	감독·심판·볼보이·관중이 전부 선수와 같은 것으로 잡힌다. 잔디 위치와 유니폼 색으로 추측해서 걸러야 한다.
축구 전용 모델 <code>models/*.pt</code>	<code>player</code> , <code>goalkeeper</code> , <code>referee</code> , <code>ball</code>	스태프·관중은 어느 클래스에도 안 잡히고, 심판·골키퍼를 모델이 직접 알려준다. 추측 단계가 통째로 사라진다.

이 구분이 중요한 이유는, 코드의 거의 모든 필터가 "모델이 안 알려주는 것을 대신 알아내는 장치"이기 때문이다. 축구 전용 모델을 쓰면 그 필터들 중 절반은 할 일이 없어진다.

3.2 추적 — 같은 선수에게 같은 번호를 유지하기

검출만으로는 "이 프레임의 3번 상자"와 "다음 프레임의 5번 상자"가 같은 사람인지 알 수 없다. ByteTrack 이 그 연결을 맡는다. 코드는 두 가지를 손봤다.

`auto_tracker_thresh` — 사용자가 정한 신뢰도의 의미를 지킨다

ByteTrack 의 내부 임계값은 0.25 로 고정되어 있다. 그래서 사용자가 `conf` 를 0.10 으로 낮춰도 0.25 미만 검출은 추적 번호를 못 받고 버려진다. 이 옵션을 켜면 임계값을 `conf` 에 맞춰 다시 계산한 트래커 설정 파일을 자동 생성해 쓴다.

match_thresh 0.9 — 작은 박스 영상에서 트랙이 끊기는 문제

코드에 적힌 실측 (20초 구간, 두 영상)

720p 영상에서 선수 키가 32px 이면 몸통 너비는 13px 남짓이다. 프레임간격 3이면 처리 간격이 0.125초라 달리는 선수가 그 사이에 자기 너비만큼 움직인다. 박스 겹침이 0 이 되어 트랙이 끊기고 새 번호를 받는다. **그 영상에서 트랙 지속 중앙값이 2장(0.2초)이었다.**

0.8 → 0.9 로 올린 뒤 :

· 사용자 720p : 추적ID 233→139개 · 트랙 지속 7→13장 · 검출 15.8→17.9명

· 전술 1080p : 추적ID 189→117개 · 트랙 지속 13→24장 · 검출 18.1→19.3명

임계값을 풀면 다른 선수로 옮겨붙을 위험이 생기는데, 그것도 따로 켜다 — 트랙 안에서 자기 키보다 크게 튼 비율이 두 영상·세 설정 모두 **0.0%** 였다. ByteTrack 이 칼만 필터 예측을 함께 쓰기 때문이다.

이 숫자를 기억해 두자. 트랙 지속이 13~24장, 즉 1~2초다. 7장에서 다시 나온다.

3.3 사람 거르기 — 경기장 밖과 화면에 박힌 그래픽

잔디 마스크 (pitch_mask)

HSV 색상 범위(기본 30~95)로 잔디를 찾아 마스크를 만들고, 가장자리를 16px 안쪽으로 좁힌 뒤, 사람 상자의 **발밑 점**이 마스크 밖이면 버린다. 30프레임마다 다시 계산하고, 리플레이·클로즈업처럼 잔디를 못 찾는 프레임에서는 직전 마스크를 유지한다.

중심점이 아니라 발밑 점을 쓰는 이유가 중요하다. 사람은 땅을 딛고 서 있으므로 지면 위치는 발이다. 중심점을 쓰면 키 큰 사람이 경기장 안에 있어도 밖으로 판정되는 일이 생긴다. 공에는 이 필터를 적용하지 않는다 — 공은 공중에 뜨기 때문이다.

정지 그래픽 트랙 (static_tracks)

하단 배너나 선수 소개 그래픽에 박힌 **사진**도 YOLO 에는 사람으로 보인다. 그런데 그건 화면에 붙박여 있어서 카메라가 움직여도 제자리다. 판정 규칙은 이렇다.

$(\text{이동 폭} \div \text{평균 키}) < 0.10$ 이고 등장 횟수 ≥ 15 프레임 → 사람이 아니라 그래픽

이동 폭을 **키로 나누는** 이유: 멀리 있어 작게 잡힌 선수도 같은 자로 재기 위해서다. 픽셀 거리를 그대로 쓰면 먼 곳의 진짜 선수가 '안 움직인 것'으로 걸린다.

3.4 팀 나누기 — 유니폼 색으로 두 팀을 가른다

심판은 경기장 안에서 뛰므로 위치로는 못 거른다. 그래서 색을 쓴다. 4단계다.

단계	하는 일	세부
①	상체 색 뽑기 torso_color()	상자에서 위 15~50% · 가운데 절반만 잘라낸다(머리와 다리를 뺀다). 그중 잔디 초록 픽셀을 빼고, 남은 픽셀의 Lab 색공간 a·b 두 채널 중앙값을 그 사람의 색으로 삼는다. 밝기(L)를 버리는 것이 핵심이다 — 그늘에 들어간 선수와 햇빛 아래 선수를 같은 색으로 묶기 위해서다.
②	표본 모으기 calibrate_teams()	영상 전체에서 균등 간격으로 16장만 뽑아 상체 색을 모은다. 12명 미만이면 팀 구분을 포기한다.
③	군집 만들기 _kmeans()	k-means 로 4덩어리를 만들고 가장 큰 두 덩어리를 양 팀으로 본다. 같은 팀이 그늘·조명 때문에 두 군집으로 쪼개지는 일이 잦아서, 남은 군집이 어느 팀 색에 "두 팀 사이 거리의 0.5배"보다 가까우면 그 팀으로 합친다. 어디에도 안 붙으면 -1 (대개 심판·골키퍼).
④	트랙 단위 확정 _decide_team()	프레임마다 따로 판정하면 한 선수의 팀이 0→-1→1 로 깜빡인다(겹침·모션블러 한 번이면 넘어간다). 분석이 끝난 뒤 track_id 별 다수결로 team 열을 한 번에 통일한다. 동물이면 -1 보다 실제 팀(0/1)을 우선한다 — 멀어서 상체가 안 잡힌 프레임이 -1 로 세어지기 때문이다.

초록 유니폼 문제 — 필터가 상체를 통째로 지우는 경우

①에서 "잔디 초록을 뺀다"는 단계가 유니폼이 초록 계열일 때 상체를 전부 지워 버린다. 실측 — 브라질(노랑) 대 멕시코(초록) 영상에서 상체 36개를 재니 36개 전부가 잔디로 판정됐다. 초록 유니폼은 당연하고, 노란 유니폼도 색조가 34 라 잔디 범위(30~95) 하한 안쪽에 들어갔다. 그 결과 팀 보정 표본이 342명 중 41명으로 줄고 96%가 미분류됐다.

그래서 torso_grass_fallback 이 있다 — 남은 픽셀이 12개 미만이면 지우지 말고 패치 전체를 쓴다. 상체 패치는 원래 대부분이 유니폼이라 잔디가 조금 섞여도 중앙값이 버틴다. 같은 영상에서 표본이 41 → 342명으로 돌아왔다.

3.5 골키퍼 — 모델이 골대 기둥을 골키퍼라고 부른다

가장 흥미로운 필터다. 축구 전용 모델은 골키퍼를 프레임마다 '골키퍼'와 '선수'로 오간다. 실측하니:

- 진짜 골키퍼 트랙은 골키퍼 라벨이 3~39%뿐이고 나머지는 선수 라벨이었다.
- 반대로 골대(세로 기둥·그물)는 골키퍼 라벨이 69~100%였다. 학습 데이터에서 골키퍼가 늘 골대 옆에 서 있어서 모델이 "기둥 = 골키퍼"로 배운 것이다.

그래서 프레임 단위로는 판정하지 않고, 골키퍼 라벨이 한 번이라도 붙은 트랙을 트랙 단위로 가른다 (resolve_keeper_tracks).

조건	판정
유니폼 색이 팀 0 또는 1 이다	필드 선수 — 골대 근처에서 잠깐 잘못 찍힌 것. 골키퍼 라벨을 선수로 되돌린다.
팀 -1 이고, 트랙 전체 신뢰도 중앙값 < 0.50	골대 — 선수 라벨 행까지 트랙을 통째로 뺀다.
그 외	골키퍼 — 트랙 전체를 골키퍼로 통일한다.

임계값 0.50 의 근거: 트랙 전체 신뢰도 중앙값이 골대는 0.22~0.41, 골키퍼는 0.60~0.81 이었다. 골키퍼 라벨이 붙은 행만 보면 0.41 / 0.45 로 거의 붙어 있는데, 진짜 골키퍼는 **선수로 찍힌 행**의 신뢰도가 높아 트랙 전체로 보면 0.41 / 0.60 으로 벌어진다. 그 사이인 0.50 에서 자른다.

3.6 공 — 여섯 단계를 거친다

이 프로그램에서 가장 공들인 부분이다. 이유는 간단하다. **공 위치가 틀리면 기여도가 통째로 틀린다**. 공이 어디 있느냐로 "누가 소유자인가", "어느 팀이 공격 중인가", "위험한 공간이 어디인가"가 전부 결정되기 때문이다.

왜 이렇게까지 하는가 — 실측 오검출

리버풀-레알 경기 50초 구간을 눈으로 전수 판정한 결과, 공 검출 **587건 중 281건(48%)이 페널티 마크**였다. 게다가 마크가 잡히는 프레임에서는 진짜 공을 대신 놓치는 일이 많았다. 무작위 104건 판정에서는 **흰 축구화가 19건(18%)**으로 가장 큰 오류였다.

① 공 전용 모델을 따로 돌린다

선수 모델은 공을 잘 못 찾는다. 공은 지름 12px 남짓이라 선수와 함께 학습된 모델에서는 늘 뒷전이다. 같은 60프레임 실측: 선수 모델 @1280 은 42장에서 발견하고 968ms/장, 공 전용 모델 @1280 은 50장에서 발견하고 134ms/장 — **더 잘 찾으면서 7배 빠르다**. 공 전용 모델의 track_id 는 1,000,000번부터 매겨 선수 번호와 섞이지 않게 한다.

② 직전 공 위치를 확대해 한 번 더 본다 (ball_focus)

먼 공이나 선수 발밑의 공은 전체 화면에서 몇 픽셀이라 신뢰도가 바닥이다. 직전 위치 주변 480px 를 잘라 960 으로 2배 키워 다시 보면 같은 공을 0.04 → 0.7~0.8 로 잡는다. 공을 놓친 지 2초가 지나면 그만 본다.

③ 네 가지 물리 검사

검사	기준	근거
크기 비율	0.16 ~ 0.36	공 박스 높이를 같은 프레임 사람 키의 중앙값 으로 나눈다. 픽셀 크기를 그대로 쓰면 '작다'와 '멀다'가 섞이는데, 마크 옆의 선수도 똑같이 멀어 작게 잡히므로 나누면 거리가 상쇄된다. 실측 분포 — 진짜 공 0.170~0.319(중앙 0.208), 페널티 마크 0.134~0.232(중앙 0.159).
밝기	최대 ≥ 215	공은 포화된 흰색이고 페인트 자국은 바랜 회백색이다. 실측 회색조 최대값 — 진짜 공 248, 페널티 마크 196, 사진기자석 216. 215 로 자르면 진짜 공을 하나도 안 잃고 마크 24건을 전부 뺀다.
모양(채움비)	≥ 0.65	흰 덩어리 넓이 $\div$ 외접 사각형 넓이. 공은 원이라 이론값 $\pi/4 \approx 0.785$, 실측 중앙값 0.77. 축구화는 끈·스트라이프 때문에 들쭉날쭉해 0.52. 종횡비로도 재봤지만 모션블러에 무너져 못 쓴다 — 채움 비율은 블러에도 버틴다 .
정지	—	제자리에 멈춰 있으면서 신뢰도가 0.30 미만인 후보는 뺀다(카메라맨·관중석·흰 축구화). 진짜 공은 결국 차므로 트랙 전체로 보면 움직인다.

밝기 검사에는 면제 규칙이 달려 있다. 날아가는 공은 모션블러로 흰색이 번져 밝기가 190~210 으로 떨어지기 때문이다. 신뢰도 0.70 이상이면 밝기 검사를 면제하되, 면제에도 바닥(180)을 두고, **움직이는 후보에게만** 면제를 준다 — 멈춘 공은 번지지 않으므로 밝게 찍히고, 페널티 마크는 제자리에 있기 때문이다.

④ 후보가 여럿이면 하나만 고른다 (BallPicker)

근거는 "경기장에 공은 하나뿐"이라는 사실이다. 직전 위치에서 등속으로 예측한 자리에 가까운 후보에게 점수를 었는다.

$$\text{점수} = \text{신뢰도} + 0.6 \times \exp(-\text{예측위치까지의 거리} \div 140\text{px})$$

페널티 마크는 늘 제자리라 예측에서 벗어나고, 진짜 공은 매끄럽게 움직여 예측에 맞는다. 실측: 프레임간 이동폭 90% 지점이 102px $\rightarrow$ 69px 로 줄었다.

스스로 강화되는 합정과 그 해법

연속성 보너스(최대 0.6)가 신뢰도 차이보다 크면 문제가 생긴다. 한번 페널티 마크에 물리면 마크는 늘 제자리라 보너스를 통째로 받고, 멀리 튜 진짜 공은 보너스를 못 받아 **영영 빠져나오지 못한다**.

실측 f342 — 진짜 공 0.79+0.00=0.79 / 페널티 마크 0.54+0.56=**1.10** $\leftarrow$ 이것이 뽐혔다.

그래서 "훨씬 확신하는 후보가 있으면 갈아탄다"(ball_switch_margin 0.20)는 장치를 뒀다. 선택이 갈린 19개 프레임을 전수 확인하니 19개 모두 갈아탄 쪽이 진짜 공이었다. 갈아탄 직후에는 **속도를 0으로 초기화**한다 — 옛 자리에서 새 자리까지의 거리를 속도로 삼으면 다음 예측이 엉뚱한 곳을 가리켜 다시 마크로 되돌아가기 때문이다.

⑤ 꺾적 잇기 (chain_ball) — 시간축으로 진짜 공을 가려낸다

한 프레임만 보면 달리는 선수의 흰 축구화는 공과 구별되지 않는다(밝기 222~250, 채움 0.62~0.83, 크기비 0.20~0.27 — 전부 공 범위). 차이는 **시간축**에 있다. 진짜 공의 약한 후보들은 서로 이어져 확실한 공으로 연결되지만,

축구화는 어떤 확실한 공과도 이어지지 않는다.

그래서 신뢰도 0.60 이상으로 잡힌 공을 끝점으로 삼아 앞뒤로, 검사에서 탈락한 후보까지 포함해 등속 예측을 따라간다. 이어진 길이가 3 이상이면 그 프레임의 선택을 바꾼다. 실측 오프라인 재현에서 교체 84건 · 추가 161건, 교체 78건을 비교하니 좋아진 것 약 50, 나빠진 것 약 5 었다.

⑥ 짧은 공백 메우기 (interpolate_ball)

앞뒤로 확실히 잡힌 위치 사이를 직선으로 잇는다. 여기서 중요한 것은 상한을 0.4초로 잡은 근거다.

빈 구간 길이	오차 중앙값 (선수 키 단위)	키의 1/3 이내 비율
0.16초	0.01	100%
0.6초	0.18	76%
0.8초	0.28	58%
1.6초	0.91	17%

처음에는 2.0초로 넣었다가 틀렸다는 것을 확인하고 0.4초로 줄였다. 공은 짧은 시간에도 방향을 바꾸고(드리블·패스) 카메라가 패닝해서 픽셀상 경로가 직선이 아니기 때문이다. 2.0초로 메운 점 52개를 눈으로 보니 제대로 맞은 것이 29%뿐이었다. 롱패스처럼 몇 초짜리 공백은 이 방법으로 메울 수 없다. 메운 행은 `interpolated=1` 로 표시한다.

3.7 출력 — tracks.csv 한 줄에 담기는 것

```
frame, time_sec, track_id, class_id, class_name, conf,
x1, y1, x2, y2,      ← 바운딩 박스 좌상단 / 우하단 (픽셀)
cx, cy,              ← 박스 중심
fx, fy,              ← 발밑 중심점 = ((x1+x2)/2, y2)  "지면 좌표 변환용"
w, h, team, interpolated
```

기여도 계산이 실제로 쓰는 것은 이 중 `frame`, `time_sec`, `track_id`, `team`, 그리고 위치로는 `fx`, `fy` 다. 나머지는 검증용(`draw_boxes`)이거나 중간 산물이다.

여기가 6장의 출발점이다

`fx`, `fy` 주석에 "지면 좌표 변환용"이라고 적혀 있지만, **tracker.py 전체에 그 변환 코드가 없다.** `homography`, `perspective`, `warp`, 미터 단위 변환을 모두 검색해도 나오지 않는다. 즉 CSV 에 적히는 것은 **끝까지 화면 픽셀 좌표**다.

4 장

측정 단계 (3) — 검증과 복구 도구

4.1 draw_boxes.py — 평균은 거짓말을 한다

이 도구는 이미 뽑아둔 tracks.csv 를 원본 영상에 다시 그려 확인용 영상을 만든다. YOLO 를 다시 돌리지 않으므로 빠르다.

```
python draw_boxes.py 영상.mp4 --from 12:30 --to 13:30 --step 3
```

빨강/파랑 = 양 팀, 주황 = 골키퍼, 보라 = 심판, 노랑 원 = 공, 회색 점선 원 = 보간으로 메운 공 위치.

이 도구가 왜 중요한가

검출 오류는 **평균 뒤에 숨는다**. "프레임당 22.7명 검출" 같은 숫자는 그 22.7명 중 셋이 관중이어도 똑같이 나온다. 오검출을 실제로 찾아내는 방법은 통계가 아니라 **하나씩 잘라서 눈으로 보는 것**뿐이다. 실제로 이 프로그램의 필터 대부분(페널티 마크 48%, 축구화 18%, 골대 기둥)은 전부 이 방식으로 발견됐다. 기여도 코드 쪽에도 같은 목적의 '표본검사' 시트가 있다(5.8 참고).

세부에도 관찰이 들어 있다. 공을 사각형이 아니라 **원**으로 그리는데, 원본에서 공 지름이 7~8px 이고 960px 로 줄이면 5px 라 선수 상자들 사이에서 보이지 않기 때문이다. 공을 마지막에 그리는 것도 같은 이유다 — "0~5초에 공을 못 찾는다"는 신고가 실제로는 검출은 정확했고 선수 라벨에 가려 안 보였던 것이었다고 주석에 적혀 있다. 공 라벨 글자를 아예 안 쓰는 것도, 글자가 공 상자보다 열 배 커서 정작 공을 가리기 때문이다.

4.2 recover_summary.py — 몇 시간짜리 분석을 지키는 장치

결과 영상을 만드는 도중에 프로그램이 닫히면 tracks.csv 는 멀쩡한데 summary.json 이 없는 상태가 된다. 그러면 다음 실행 때 '이미 한 건 건너뛰기'가 작동하지 않아 몇 시간짜리 분석을 처음부터 다시 돌게 된다. 이 스크립트는 CSV 를 한 줄씩 흘려 읽어(큰 파일도 메모리를 안 먹는다) 통계를 다시 계산하고 summary.json 을 만든다. CSV 만으로 알 수 없는 항목(경기장 밖 제거 수 등)은 `null` 로 두고, `recovered` 필드에 복구본임을 적어 둔다.

5 장

해석 단계 — 기여도는 이렇게 계산된다

5.1 무엇을 재려는 것인가

골·어시스트·패스 성공률은 **공에 닿은 사람**만 기록한다. 그런데 축구에서 일어나는 일의 대부분은 공 없이 일어난다. 수비수가 공간을 미리 막아서 패스 선택지를 지운 것, 미드필더가 소유자에게 붙어 시간을 빼앗은 것, 공격수가 좋은 자리에 서 있어서 수비를 끌고 간 것 — 이런 것은 어떤 기록지에도 안 남는다. 이 프로그램이 재려는 것이 그것이고, 세 축으로 나눈다.

지표	무엇을	언제 나오나	한 줄 정의
SC	공간 통제	팀이 수비할 때	위험한 공간을 상대보다 먼저 덮고 있었는가
PR	압박 기여	팀이 수비할 때	공 가진 상대에게 얼마나 붙었고 얼마나 빠르게 좁혔는가
PA	패스 유인	팀이 공격할 때	동료가 나에게 패스하고 싶을 만큼 좋은 선택지였는가
DPI	수비 합성	—	$0.5 \cdot z(SC) + 0.5 \cdot z(PR)$
TC	총 기여	—	세 지표를 순위 정규화해 1/3씩 합친 값

SC·PR 과 PA 는 서로 다른 프레임에서 나온다

이 점이 결과표를 읽을 때 계속 걸린다. 우리 팀이 공을 갖고 있는 동안 우리 선수에게는 PA 만 쌓이고 SC·PR 은 쌓이지 않는다. 그래서 **양 팀이 공수를 주고받는 구간**이 충분히 있어야 세 지표가 다 모인다. 짧은 클립에서 한 팀이 계속 공을 갖고 있었다면 한쪽은 수비 기록이, 다른 쪽은 공격 기록이 아예 없어서 TC 를 낼 수 없다. 지난번 결과표에서 TC 가 대부분 비어 있던 이유 중 하나다.

5.2 계산 전 준비 — 좌표를 믿을 수 있게 만드는 과정

수식에 들어가기 전에 코드가 하는 일이 꽤 많다. 순서대로다.

순서	단계	내용
①	자체 검증 12항목	계산이 정의대로 되는지 먼저 확인한다. 등속 5 m/s 가 정확히 5.00 으로 나오는지, Shapley 배분의 합이 총량과 일치하는지(효율성 공리), 위험도 가중치의 방향이 맞는지 등. 하나라도 실패하면 거기서 멈춘다 — 틀린 계산으로 낸 표를 보여주지 않기 위해서다.
②	좌표계 판정	원점이 구석인지 중앙인지 자동 판정한다. 판정 기준은 좌표의 중앙값 이다. 최소값으로 판정하면 스로인하러 라인 밖에 선 선수 한 명 때문에 판정이 뒤집히고, 전체 좌표가 반 경기장만큼 밀린 채로 조용히 계산된다.
③	경기장 밖 사람 제거	경계에서 3m 넘게 밖에 있는 행을 버린다(감독·사진기자·안전요원). 스로인·코너킥 선수는 실제로 라인 밖에 서므로 여유를 둔다.
④	공격 방향 판정	골키퍼는 자기 골대 앞에 상주하므로, 팀별로 가장 낮은 X 에 상주하는 선수 를 비교해 어느 팀이 x=0 골대를 지키는지 알아낸다. 이 값이 틀리면 위험도 가중치가 상대 골대에 걸리고 PA 의 전진 부호가 뒤집혀 결과가 통째로 무의미해진다. 전·후반이 한 파일에 있으면 방향이 중간에 바뀌므로 그것도 검사해 경고한다.
⑤	ID 재연결	ID 기준 이동속도의 95% 지점이 15 m/s 를 넘으면 "프레임마다 ID 가 뒤바뀌는 데이터"로 보고 헝가리안 매칭으로 다시 이어 붙인다. 사람이 갈 수 있는 거리를 넘는 연결은 거부한다. 7장에서 이 단계가 문제를 일으킨다.
⑥	결측 보간	선수 좌표는 0.3초, 공 좌표는 0.2초 이하의 끊김만 선형보간으로 메운다. 메운 행은 표시해 둔다.
⑦	속도 계산	중앙차분(앞뒤 3프레임)으로 속도를 구하고, 11 m/s(=39.6 km/h)를 넘으면 이상치로 버린다. 프로 경기 실측 최고속도가 34~37 km/h 이므로 그 위는 추적 오류다.

5.3 공 소유 판정 — 모든 계산의 기준점

프레임마다 "지금 누가 공을 갖고 있나 / 어느 팀이 공격 중인가"를 정한다. 이것이 정해져야 SC 의 수비팀, PR 의 목표물, PA 의 패스 시작점이 정해진다.

- 가장 가까운 선수**를 찾는다. 그 거리가 POSS_RADIUS (데이터를 보고 2~8m 중 자동 결정) 이내면 소유자로 인정한다.
- 데드볼 제외** — 공이 라인 밖 1m 를 넘어가면 소유자를 두지 않는다. 90분 경기에서 공이 실제로 살아 있는 시간은 55~60분뿐이고, 스로인을 기다리며 자리 잡는 배치를 전술적 공간 다툼으로 계산하면 안 되기 때문이다.
- 공중볼 제외** — 공 높이 열이 있고 1.5m 를 넘으면 소유자를 두지 않는다. 크로스·헤더 경합은 점프와 타이밍으로 결정되므로 지면 2D 거리로는 소유자를 정할 수 없다.
- 전환 인정 조건** — 소유팀이 바뀌었다고 바로 인정하지 않는다. 1.5초(HOLD_FRAMES)보다 짧으면서 **앞뒤가 같은 팀**인 구간은 잡음으로 보고 흡수한다. A→B→A 의 짧은 B 는 지우고, A→B→C 의 B 는 실제 전환이므로 남긴다.
- usable 판정** — 소유팀이 정해졌고 **양 팀 모두 최소 인원(명목 인원의 80%)이 추적된 프레임만** 계산에 쓴다.

usable 이 왜 필요한가

SC 는 경기장 전체 격자를 "가장 가까운 수비수"에게 나눠 주는 계산이다. 뒤쪽 수비수가 화면 밖으로 빠지면 남은 수비수가 그 빈 자리의 공간 점유까지 대신 받아 SC 가 부풀려진다. 그래서 인원이 모자란 프레임은 아예 계산하지 않는다. 한 대의 중계 카메라로 만든 좌표는 화면에 경기장의 2/3 정도만 들어오므로, usable 비율이 낮게 나오는 것이 오히려 정직한 신호다.

5.4 SC — 공간 통제 (Shapley 배분)

세 걸음으로 이루어진다.

1걸음 · 경기장을 격자로 자르고 칸마다 위험도를 매긴다

$$w(\text{칸}) = \exp(-\text{골대까지거리} \div 18) \times \exp(-\text{공까지거리} \div 20) \times \text{칸넓이}$$

1m × 1m 격자이므로 105 × 68 경기장이면 약 7,140칸이다. "위험하다"는 것은 내 골대에 가깝고 동시에 공에 가깝다는 뜻이다. 둘 중 하나만 만족하는 칸은 값이 급격히 떨어진다. 18m, 20m 는 감쇠 거리로, 그만큼 멀어질 때마다 값이 1/e(약 37%)로 준다.

예: 내 골대에서 20m, 공에서 15m 떨어진 칸 → $\exp(-20/18) \times \exp(-15/20) = 0.329 \times 0.472 = 0.156$. 같은 거리라도 골문 바로 앞(골대 5m, 공 5m)이면 $0.757 \times 0.779 = 0.590$ 으로 네 배 가까이 크다.

2걸음 · 각 칸을 누가 통제하는지 정한다

그 칸까지의 거리가 가장 가까운 공격수보다 가까운 수비수들이 그 칸을 통제한다고 본다. 한 명일 수도, 세 명일 수도, 아무도 없을 수도 있다(그 칸은 공격팀 영역이므로 아무에게도 안 간다).

3걸음 · 통제하는 수비수끼리 나눈다 (Shapley)

칸을 함께 덮은 수비수가 k 명이면, 각자 $w(\text{칸}) \div k$ 씩 가져간다

협력게임 이론의 Shapley 값에서 대칭성과 효율성을 만족하는 배분이다. **효율성**: 한 프레임에서 모든 수비수가 받은 SC 의 합은 통제된 총 위험도와 정확히 같다(코드가 매번 계산한다). **대칭성**: 같은 칸을 같은 조건으로 덮은 두 수비수는 같은 값을 받는다. 덕분에 "혼자 넓게 막은 수비수"와 "둘이 겹쳐 막은 수비수"가 공정하게 평가된다.

프레임마다 위험도 총량이 크게 다르기 때문에(공이 자기 박스 안이면 총량 320, 상대 박스 쪽이면 29 로 11배 차이), 원값 SC 와 함께 팀 총량 대비 몫인 SC_share 도 낸다. 효율성 공리 덕분에 한 프레임의 share 합은 정확히 1 이다.

5.5 PR — 압박 기여

목표물은 상황에 따라 다르다. 소유자가 정해진 프레임(settled)에서는 볼 소유자, 정해지지 않은 프레임(transition)에서는 공이다. 수비팀 선수마다 목표물까지의 거리 d 와 접근속도를 재서 두 항으로 합친다.

$$\text{근접도} = \text{clip}(1 - d \div 10m, 0, 1)$$

$$PD(\text{거리항}) = \text{근접도}^2 \quad PV(\text{속도항}) = \text{clip}(\text{접근속도} \div 5, 0, 1) \times \text{근접도}$$

$$PR = 0.6 \times PD + 0.4 \times PV$$

접근속도는 거리의 중앙차분에 음수를 붙인 값이다(가까워지면 +). 근접도를 제공하는 것은 "붙어 있는 것"에 확실히 더 큰 값을 주기 위해서다.

속도항에도 근접도를 곱하는 이유 — 고쳐진 버그

원래 속도항은 $\text{clip}(\text{접근속도}/5, 0, 1)$ 뿐이라 **거리가 전혀 안 들어갔다**. 그래서 공에서 40m 떨어진 수비수가 공 쪽으로 5 m/s 로 달리지만 해도 $PV=1 \rightarrow PR=0.4$ 가 나왔고, 이것은 볼 소유자에게 1m 까지 붙어 자리를 지킨 수비수 ($PR=0.486$)와 거의 같은 값이었다.

모사 경기에서 재 보니 **25m 밖에서 쌓인 값이 PR 총량의 50%**였고, 선언한 가중치는 60:40 인데 실제 기여는 거리 18 : 속도 82 였다. 먼 거리에서의 복귀 주력은 '압박'이 아니라 '활동량'이다. 지금은 속도항에도 같은 근접도를 곱해 10m 밖에서는 두 항 모두 0 이 된다.

상황	PD	PV	PR	해석
소유자에게 3m, 2 m/s 로 접근	0.49	0.28	0.406	제대로 된 압박
소유자에게 1m, 정지(자리 지킴)	0.81	0.00	0.486	붙어서 막고 있음
30m 밖에서 5 m/s 로 복귀	0.00	0.00	0.000	압박이 아니라 활동량

결과표에는 PR 평균 외에 두 열이 더 나온다. **PR_zone** 은 "압박권(10m) 안에 있던 프레임 비율"(관여의 **양**), **PR_in** 은 "관여했을 때의 평균 PR"(관여의 **질**)이다. PR 평균 하나에는 이 둘이 섞여 있어서 수비형 미드필더(양이 많음)와 센터백(양이 없음)이 구분되지 않기 때문이다.

5.6 PA — 패스 유인도

공을 **안 가진** 공격팀 선수마다, "지금 소유자가 나에게 패스한다면 얼마나 좋은 선택인가"를 세 축으로 잴다.

성분	정의	의미
ℓ 길목 개방도	$\text{clip}(\text{패스선까지의 최소 수직거리} \div 3m, 0, 1)$	소유자와 나를 잇는 직선을 수비수가 얼마나 막고 있다. 수선의 발이 선분 안에 있는 수비수만 센다 — 뒤쪽이나 너머의 수비수가 길을 막는 것으로 잘못 계산되는 것을 막는다.
φ 수신 공간	$\text{clip}(\text{가장 가까운 수비수까지 거리} \div 8m, 0, 1)$	공을 받았을 때 자유로운가.
π 전진 가치	$0.3 + 0.7 \times (\text{clip}(\text{이득} \div 30m, -1, 1) + 1) \div 2$	이득 = (소유자의 골대까지 거리) - (내 골대까지 거리). 후방 패스도 0 은 아니므로 하한 0.3.

$$PA = l^a \times \phi^b \times \pi^c \quad (\text{기본값 } a = b = c = 1)$$

곱셈 모델이라 **하나라도 0 이면 전체가 0**이다. 아무리 자유롭고 전진하는 자리라도 패스길이 막혀 있으면 선택지가 아니라는 뜻이고, 이는 축구의 상식과 맞는다.

예: 길목 1.5m(=0.5), 가장 가까운 수비수 4m(=0.5), 전진 이득 15m(=0.825) → $PA = 0.5 \times 0.5 \times 0.825 = 0.206$

전진 가치를 X 좌표가 아니라 '골대까지 거리'로 재는 이유

X 성분만 보면 측면으로 넓게 벌리는 전환 패스(스위치)나 대각 패스가 전부 '전진 0'으로 깎인다. 실제 빌드업의 상당 부분이 그 대각·측면 전환으로 이뤄지고, 골문에서 먼 측면으로 가는 패스와 골문 정면으로 파고드는 패스는 가치가 분명히 다르다. 골대까지의 유클리드 거리 감소로 재면 두 경우가 자연스럽게 구분된다.

지수 a, b, c 를 실제 패스에서 학습하는 장치

곱셈 모델에 로그를 씌우면 선형모델이 되므로, 실제로 일어난 패스를 정답 삼아 로지스틱 회귀로 지수를 추정할 수 있다. 양성은 "그 프레임에 실제로 공을 받은 선수", 음성은 "같은 프레임의 나머지 아군"이다. 그런데 채택 조건이 엄격하다.

- 패스 표본이 **200개 이상**이어야 학습을 시도한다.
- 패스 단위로 70/30 분할해 **학습에 안 쓴 패스로** 성능(AUC)을 잰다. 검증용 패스가 50개 미만이면 포기한다.
- 그것도 모자라 **순열검정**을 한다 — 정답을 무작위로 섞었을 때의 AUC 개선 분포를 60회 만들고, 실제 개선이 그 분포의 95% 지점을 넘을 때만 채택한다.
- 길목 계수가 음수로 나오면(신호가 뒤집혔다는 뜻) 무조건 기각한다.

이 장치가 막아 주는 것

예전 문턱은 30개였는데, "패스 상대가 무작위인" 모사 데이터에서 패스 44개로 AUC 0.387 → 0.613 이 나와 순열검정 문턱까지 넘고 채택된 적이 있다. **존재하지 않는 신호를 학습한 것이다.** 한 번 채택되면 모든 PA 값이 그 지수로 다시 계산되므로 TC 순위까지 잡음이 된다. 90분 경기의 팀 패스는 보통 400~700개라 200개는 무리한 요구가 아니다.

5.7 DPI · TC · TC_adj — 합치기

세 지표는 단위도 분포도 다르고(SC 는 위험도 가중 면적, PR·PA 는 0~1), 나오는 프레임도 다르다. 그래서 원값을 더하는 것은 의미가 없고 같은 자로 맞추 뒤 더해야 한다.

$$DPI = 0.5 \cdot z(SC) + 0.5 \cdot z(PR) \quad (\text{수비 합성})$$

$$TC = \frac{1}{3} \cdot n(SC) + \frac{1}{3} \cdot n(PR) + \frac{1}{3} \cdot n(PA) \quad n(\) = \text{순위 정규화}$$

z 가 아니라 순위 기반을 기본으로 쓰는 이유: SC 는 좌우대칭인데 PR 은 오른쪽으로 길게 늘어져 있다 (왜도 +1.2, 최대/중앙 6.5). z 로 더하면 꼬리가 긴 PR 의 극단값이 합계를 끌고 가서, 가중치는 1/3씩인데 실제 영향력은 그렇지 않게 된다.

합치기 전에 빼는 것

- **골키퍼** — 위험도가 최대인 골문 앞에 상시 서 있어서 가만히 있어도 SC 를 쓸어담는다(모사 경기에서 SC 3위/7위). 자기 골대까지의 평균거리가 22m 미만인 '팀당 한 명'을 골키퍼로 본다(실측: 골키퍼 ~20m, 센터백 ~31m). 검출기가 골키퍼를 알려주면 그것을 우선한다.
- **표본이 너무 적은 선수** — 팀 중앙값의 20% 미만으로 관측된 선수는 뺀다. 평균이 튀는 것도 문제지만, 더 큰 문제는 그 값이 **정규화 기준까지 흔들여 남의 점수를 바꾼다**는 것이다.

TC_adj — 포지션 보정

TC 는 '어느 자리에서 뛰는가'와 상관성이 크다. 실측 순위상관은 SC 가 $\rho = -0.87$ (뒤에 설수록 높음), PR 이 $+0.68$, PA 가 $+0.29$ 였다. 보정 없이는 순위가 능력이 아니라 포지션을 반영한다. 그래서 TC 를 '자기 골대까지의 평균거리'로 회귀한 **잔차**를 TC_adj 로 함께 낸다.

두 숫자를 읽는 법

TC = 팀에 준 총량. **TC_adj** = 같은 위치의 선수들과 비교했을 때.

둘 중 하나가 정답이 아니다. **둘이 많이 다르면 그 순위는 포지션이 만든 것**이라는 뜻으로 읽으면 된다. 코드는 TC 의 몇 %가 거리 하나로 설명되는지(R^2)를 매번 출력하고, 40%를 넘으면 경고한다.

가중치에 대한 태도

0.5/0.5 와 1/3씩은 모두 "검증 전 기준선"이다. 어느 하나가 더 중요하다는 선험적 근거가 없고, **기존 선수 순위에 맞춰 가중치를 조정하면 결론을 먼저 정해 놓고 식을 맞추는 것**이 되기 때문이다. 대신 코드는 실행할 때마다 "가중치를 바꿨을 때 순위가 얼마나 흔들리는지"를 출력한다. 그 값이 낮으면($p < 0.5$) 가중치가 순위를 지배하고 있다는 뜻이므로, 승패·득점 같은 결과 라벨이 생기면 반드시 다시 정해야 한다.

5.8 진단 출력 읽는 법

이 코드는 숫자만 내지 않고, **그 숫자를 믿어도 되는지**를 매 실행마다 같이 낸다. 순서대로 무엇을 보는지다.

구간	확인할 것
[검증]	12항목이 모두 PASS 인가. 하나라도 FAIL 이면 아래 숫자는 볼 필요가 없다.
[읽기]	좌표 범위가 경기장과 맞는가 · 공격 방향 자동 판정이 맞는가 · 프레임당 인원이 명목 인원의 90% 이상인가 · ID 재연결 후 ID 개수가 선수 수와 비슷한가.
[계산]	usable 비율. 이것이 30% 미만이면 원인이 '소유자 미판정'인지 '추적 인원 부족'인지도 같이 나온다. 앞쪽이 크면 공 검출 문제, 뒤쪽이 크면 화면 밖 선수 문제다.
[지표 진단]	바닥 효과 (대부분 0 이라 사실상 횡수만 재는 것)와 포화 (대부분 1 이라 정보를 못 나르는 것)를 본다. 포화가 절반을 넘으면 그 차원은 순위에 기여하지 못하므로 상수를 키워야 한다. 분할 재현성 (앞구간과 뒷구간 순위상관)도 낸다 — 낮으면 선수가 아니라 상황을 재고 있는 것이다.
[총 기여도]	무엇을 뺐는지(골키퍼·표본부족), TC 의 몇 %가 포지션인지, 가중치를 바꾸면 순위가 얼마나 흔들리는지.
표본검사 시트	선수마다 점수를 가장 많이 끌어올린 프레임의 영상 시각(mm:ss) 을 뽑아 준다. 최소 5초 간격으로 골라 서로 다른 장면이 나오게 한다. 영상에서 그 지점 몇 개만 열어 보면 표 전체를 믿어도 되는지 금방 알 수 있다. — draw_boxes.py 와 같은 철학이다.

6 장

두 단계 사이 — 지금 끊어져 있는 다섯 곳

여기부터는 코드를 읽기만 한 것이 아니라 **실제로 돌려서 확인한** 내용이다. tracker.py 가 내는 CSV 열 목록을 기여도 코드의 열 인식 함수에 그대로 넣어 보고, 공 판정 함수에 실제 값을 넣어 봤다.

6.1 좌표가 픽셀이다 — 미터로 바꾸는 코드가 어디에도 없다 가장 심각

CSV 의 `fx`, `fy` 주석에는 "지면 좌표 변환용"이라고 적혀 있다. 그런데 **변환 자체가 구현되어 있지 않다**. tracker.py 2,593행 어디에도 homography · perspective · warp · 미터 변환이 없다.

기여도 코드는 픽셀을 받으면 스스로 멈춘다.

좌표 범위가 경기장보다 훨씬 크다 (X 0~1920, Y 0~1080).
화면 픽셀 좌표로 보인다. 호모그래피로 미터 좌표로 변환한 뒤에 넣어야 한다.
픽셀로 계산하면 화면 위/아래의 1m 가 서로 달라져서 결과가 전부 무의미해진다.

왜 픽셀로는 절대 안 되는가

카메라는 경기장을 비스듬히 본다. 그래서 **화면 아래쪽(가까운 곳)의 100픽셀과 위쪽(먼 곳)의 100픽셀이 서로 다른 거리다**. 가까운 쪽에서는 5m 일 수 있고 먼 쪽에서는 20m 일 수 있다.

기여도 계산의 거의 모든 상수가 미터 기준이다 — 격자 1m, 위험도 감쇠 18m·20m, 압박권 10m, 패스 길목 3m, 수신 공간 8m, 소유 반경 2~8m, 최고속도 11 m/s. 픽셀을 넣으면 이 상수들이 화면 위치에 따라 제멋대로 늘어났다 줄었다 한다. **계산은 조용히 끝나지만 숫자는 전부 무의미하다**.

6.2 열 이름이 맞지 않는다 확인함

tracker.py 의 `CSV_COLUMNS` 를 기여도 코드의 `map_columns()` 에 그대로 넣고 실행한 결과다.

```
열 인식: frame ← 'frame' , time_s ← 'time_sec' , track_id ← 'track_id' ,  
        conf ← 'conf' , team ← 'team'  
  
인식된 표준 열 : ['frame', 'time_s', 'track_id', 'conf', 'team']  
필수 열 중 없는 것 : {'X', 'Y'} ← 여기서 멈춘다
```

기여도 코드의 별칭 목록에 `x`, `pos_x`, `pitch_x` 같은 이름은 있지만 `cx` · `fx` 는 없다. 그래서 위치 열을 하나도 못 찾고 "필수 열이 없다: {'X','Y'}" 로 중단된다. `class_name` · `class_id` 도 마찬가지로 `cls` 로 인식되지 않는다.

해결은 간단하다 — 변환 단계에서 `fx` → `X`, `fy` → `Y` 로 이름만 바꿔 주면 된다(단위는 6.1 과 별개 문제).

6.3 팀 번호 규약이 다르다

tracker 는 0 / 1 / -1 (심판·골키퍼 등) / 빈칸(공)을 쓴다. 기여도 코드는 **팀이 정확히 2개**여야 하고, 심판으로 인식하는 키워드는 `ref`, `심판`, `none`, `nan`, `unknown` 이다. -1 은 이 목록에 없으므로 **세 번째 팀**으로 세어지고,

다음에서 멈춘다.

팀이 2개여야 하는데 ['0', '1', '-1'] 이다. team 열을 정리할 것.

해결: tracker 쪽 `drop_non_team` 을 켜거나, 변환 단계에서 -1 행을 빼면 된다. 다만 골키퍼가 -1 로 묶여 있다면 함께 빠진다는 점에 주의해야 한다 — 골키퍼가 빠지면 골문 앞 공간을 센터백이 대신 받아 SC 가 부풀려진다. 축구 전용 모델을 쓰면 골키퍼가 별도 클래스로 나오므로, 골키퍼는 남기고 심판만 빼는 처리가 필요하다.

6.4 공 행 인식 조건부 작동

기여도 코드의 공 판정 함수에 실제 값을 넣어 본 결과다.

넣은 값	공으로 인식?	어떤 경우인가
'ball'	✓ 인식	축구 전용 모델의 클래스 이름
'sports ball'	✗ 못 함	COCO 모델의 클래스 이름 — 정확히 'ball' 이어야 인식된다
1000001	✓ 인식	공 전용 모델이 붙인 track_id (100만번대)
32 (COCO 공 클래스번호)	✗ 못 함	—

다행인 우연

tracker.py 의 `ball_id_base = 1_000_000` 과 기여도 코드의 `BALL_ID_BASE = 1_000_000` 이 값이 같다. 그래서 **공 전용 모델을 쓰면 track_id 만으로 공 행이 정확히 인식된다**. 반대로 COCO 모델의 `sports ball` 만 있으면 공을 찾지 못해 "공 좌표를 찾지 못했다" 로 중단된다. → **공 전용 모델을 반드시 쓸 것.**

6.5 interpolated 열이 무시된다 경미

tracker 는 보간으로 메운 공 위치를 `interpolated=1` 로 정직하게 표시한다. 그런데 기여도 코드는 그 열을 읽지 않고, 자기만의 보간을 따로 한다(0.2초 이하). 결과적으로 **추정 위치와 측정 위치가 똑같은 자격으로 소유자 판정에 쓰인다**. 치명적이지는 않지만, "어느 선수의 기여가 실제 측정에서 나왔고 어느 것이 추정에서 나왔는지" 구분할 수단이 사라진다.

7 장

사례 해부 — 지난번 결과표가 망가진 이유

앞서 보여주신 결과표에는 세 가지 증상이 있었다. 셋 다 같은 뿌리에서 나온다.

증상 ① — 선수가 22명인데 표에 160행이 나왔다

track_id	team	frames	SC_mean	PR_mean	PA_mean	TC
Red_new102_12	Red	13	1.6588	0.0000	0.6510	0.1690
Red_new102_8	Red	13	1.0451	0.2507	0.1722	0.0590
Blue_new193_9	Blue	1	NaN	NaN	0.0181	NaN
Blue_new170_11	Blue	1	NaN	NaN	0.0833	NaN
...						

(160행 계속)

Red_new102_12 라는 이름은 기여도 코드가 붙인 것이다. 형식은 팀_new{프레임}_{번호} — "102번 프레임에서 새로 생긴 12번 궤적"이라는 뜻이다. 즉 한 선수의 궤적이 프레임마다 토막나 있다. 원인 사슬은 이렇다.

원인 1 · ByteTrack 이 주는 track_id 가 오래 못 간다

tracker.py 주석의 실측값: 720p·프레임간격 3 에서

트랙 지속 중앙값 2장(0.2초)

, match_thresh 를 0.9 로 올려도 7→13장(0.5초). 선수 한 명이 20초 동안 수십 개의 번호를 갈아탄다.

원인 2 · 기여도 코드의 ID 재연결이 '새 번호 발급'으로 대응한다

헝가리안 매칭으로 앞 프레임과 이어 붙이되, 이어지지 않는 검출에는 새 이름을 준다. 한 번 끊긴 궤적은 다시 만나지 않는다

— 몇 프레임 뒤 같은 자리에 다시 나타나도 별개의 선수가 된다.

결과 · 22명이 160개 ID 로 부서진다

모사 데이터로 같은 상황을 재현하니 22명 →

948개 ID

까지 부서졌다. 이 상태에서는 '선수별' 표가 선수를 가리키지 않는다.

이것은 수식의 문제가 아니다

SC · PR · PA 자체는 정상 작동했다(효율성 공리 검증도 통과했다). 무너진 것은 "같은 선수"라는 개념이다. 그래서 프레임 단위 결과(SC/PR/PA 분포)까지는 의미가 있어도 선수 순위는 의미가 없다. 기여도 코드도 이것을 알고 경고를 출력한다 — "조각난 ID 로는 '선수별' 표가 선수를 가리키지 않는다."

증상 ② — TC 가 대부분 비어 있다(NaN)

두 가지가 겹쳤다.

1. 구조적 이유 — 표를 보면 Red 쪽에는 SC·PR 은 있는데 PA 가 없고, Blue 쪽에는 PA 만 있고 SC·PR 이 거의 없다. 관측 구간 내내 Blue 가 공을 갖고 Red 가 수비했다는 뜻이다. TC 는 세 지표가 다 있어야 계산되므로 양쪽 다 널

수가 없다. 더 긴 구간, 공수가 오가는 구간이 필요하다.

2. **파편화 이유** — 1~2프레임만 존재하는 조각은 셋 중 하나만 걸리기 쉽다. 160행 중 대부분이 `frames=1` 이다.

증상 ③ — PR 이 거의 전부 0.0000 이다

PR 은 압박권 10m 밖에서는 정확히 0 이 된다(5.5 참고). 즉 **수비수 중 누구도 볼 소유자의 10m 안에 없었다**는 뜻이다. 진단 출력에도 그렇게 찍혀 있었다.

PR 압박권($d \leq 10m$) 안 0.0% / 공까지 거리 중앙값 50.1 m

공까지 거리 중앙값이 50m 라는 것은 축구 경기에서 나올 수 없는 값이다. 이것은 압박을 안 했다는 뜻이 아니라 **좌표계가 틀렸다는 신호**다 — 6.1 의 픽셀/미터 문제이거나, 공 좌표와 선수 좌표의 정합이 깨진 경우다.

같이 보였던 신호들

출력	의미
usable 4%	96%의 프레임이 계산에서 빠졌다. 원인은 '소유자 미판정 60% / 추적 인원 부족 91%' 로 함께 찍혔다. 뒤쪽이 크므로 화면 밖 선수가 빠진 좌표 가 주원인이다.
프레임당 인원이 명목 인원의 73%	22명 중 16명만 잡혔다. 한 대의 카메라로는 경기장의 2/3 만 들어온다.
골키퍼를 1명만 찾았다	ID 가 파편화되어 '자기 골대 앞에 상주하는 한 명'을 찾을 수 없었다. 남은 골키퍼가 SC 상위권을 차지한 채 순위에 들어간다.
SC_mean 24.5 (frames=6)	6프레임만 존재한 조각이 SC 1위다. 수비수가 몇 명 안 잡힌 프레임에서 그 조각이 빈 자리의 공간 점유까지 받아간 결과다.

8 장

고치는 순서

네 가지를 이 순서로 해야 한다. 순서가 중요하다 — 앞의 것을 안 고치고 뒤의 것을 고치면 헛수고가 된다.

1순위 · 픽셀 → 미터 변환 (호모그래피)

이것 없이는 나머지가 전부 무의미하다. 방법은 경기장의 **알려진 지점 4개 이상**을 화면에서 찍어 두 평면 사이의 변환 행렬을 구하는 것이다.

```
import cv2, numpy as np
# 화면에서 찍은 점 (픽셀)           실제 경기장 위치 (미터)
src = np.float32([[412, 880], [1503, 872], [1740, 402], [188, 410]])
dst = np.float32([[ 0, 0], [ 105, 0], [ 105, 68], [ 0, 68]])
H, _ = cv2.findHomography(src, dst)

pts = np.float32(df[["fx", "fy"]].values).reshape(-1, 1, 2)
xy = cv2.perspectiveTransform(pts, H).reshape(-1, 2)
df["X"], df["Y"] = xy[:, 0], xy[:, 1]
```

쓸 만한 기준점: 경기장 네 모서리, 페널티박스 네 모서리, 골에어리어 모서리, 하프라인과 터치라인이 만나는 두 점, 센터서클 좌우 끝.

중계 영상이면 이것만으로 부족하다

위 방법은 **카메라가 고정일** 때만 한 번으로 끝난다. 중계 카메라는 패닝·줌·틸트를 하므로 **프레임마다 변환 행렬이 달라진다**. 이 경우 선택지는 셋이다.

- ① **고정 전문 카메라**로 촬영한 영상을 쓴다 — 가장 쉽고 확실하다.
 - ② 프레임마다 **경기장 선을 검출**해 변환을 다시 계산한다(허프 변환 등).
 - ③ 경기장 키폰트를 직접 예측하는 모델을 붙인다(SoccerNet 계열 공개 모델이 있다).
- ①이 가능하다면 ①을 권한다. 기여도 계산은 어차피 경기장 전체가 보이는 좌표를 요구하는데, 중계 화면은 2/3 만 보여 주기 때문이다(7장의 usable 4% 참고).

2순위 · 추적 지속 늘리기 — 목표는 "한 선수 = 한 ID"

손볼 것	왜
<code>vid_stride</code> 를 1 또는 2 로	3이면 처리 간격 0.125초에 선수가 자기 몸 너비만큼 움직여 박스 겹침이 0 이 된다. 파편화의 최대 원인.
<code>imgsz</code> 를 960 이상으로	박스가 커지면 겹침 계산이 안정된다. 축구 모델은 검출 수는 안 늘지만 추적 안정성은 좋아진다.
<code>track_buffer</code> 를 60~90 으로	가려진 선수를 더 오래 기억한다. 기본 30은 원본 프레임 기준이라 stride 3이면 실질 10프레임뿐이다.
<code>bytetrack_recover_low</code> 켜기	낮은 신뢰도 검출을 트랙 잇기 용도로만 쓴다. 가려졌다 나온 선수의 ID 가 덜 끊긴다.

확인 방법: 20초 구간을 돌린 뒤 `summary.json` 의 `unique_track_ids` 를 본다. 22명 경기에서 이 값이 30 안쪽이면 성공, 100을 넘으면 아직 파편화 상태다. `draw_boxes.py --labels id` 로 영상을 만들어 번호가 얼마나 자주 바뀌는지 눈으로 보는 것이 가장 확실하다.

3순위 · 변환 스크립트 (tracks.csv → 기여도 입력)

6장의 나머지를 한 번에 처리하는 30줄짜리 스크립트다. 하는 일은 다섯 가지다.

1. 공 행(`track_id` $\geq 1,000,000$)을 따로 떼어 `ball_x`, `ball_y` 로 만든다
2. 사람 행에서 심판(`team` = -1 이면서 골키퍼가 아닌 행)을 뺀다
3. `fx`, `fy` 에 호모그래피를 적용해 `x`, `y` (미터)를 만든다
4. `team` 0/1 을 알아보기 쉬운 이름으로 바꾼다 (예: Home / Away)
5. 골키퍼 `track_id` 목록을 뽑아 둔다 → 기여도 코드의 `GK_IDS` 에 직접 넣으면 자동 추정보다 정확하다

4순위 · 기여도 코드 쪽 정리

1~3을 해도 완벽한 추적은 없으므로, 남은 파편을 해석 단계에서 흡수하는 장치가 있으면 좋다.

- **꺾적 잇기(track stitching)** — 시간이 겹치지 않으면서 끝점과 시작점이 시공간적으로 이어지는 조각들을 하나로 합친다. 끊긴 뒤 다시 나타난 선수를 되찾는 단계로, 지금 코드에는 없다.
- **표 정리** — 관측이 너무 적은 조각은 표에서 빼고(엑셀에는 남기고), NaN 을 - 로 표시하고, 화면에는 22행 남짓만 보이게 한다.
- **TC 를 부분 지표로도 계산** — 세 지표가 다 없으면 지금은 NaN 인데, 있는 것끼리 가중치를 다시 나눠 계산하고 "무엇으로 계산했는지"를 함께 표시하면 짧은 클립에서도 쓸 수 있다.

9 장

용어 사전

용어	뜻
검출 (Detection)	한 장의 이미지에서 "여기에 사람이 있다"를 찾는 것. 프레임 사이의 관계는 모른다.
추적 (Tracking)	프레임을 가로질러 "이 사람과 저 사람이 같은 사람"임을 잇는 것. 그 결과가 track_id.
ByteTrack	검출 결과를 잇는 추적 알고리즘. 칼만 필터로 다음 위치를 예측하고, 박스 겹침(ioU)으로 짝을 짓는다. 낮은 신뢰도 검출도 2단계로 활용하는 것이 특징이다.
IoU	두 박스의 겹친 넓이 ÷ 합친 넓이. 1이면 완전히 같고 0이면 안 겹친다.
헝가리안 매칭	N개와 M개를 짝지을 때 전체 비용의 합이 최소가 되는 짝을 찾는 알고리즘. 추적에서 표준으로 쓰인다.
호모그래피	한 평면(화면)의 점을 다른 평면(경기장)의 점으로 옮기는 3×3 변환 행렬. 기준점 4개면 구할 수 있다.
Shapley 값	여럿이 협력해 만든 성과를 각자의 기여도에 따라 나누는 협력게임 이론의 배분 방식. 합이 보존되고 (효율성), 같은 기여는 같은 몫을 받는다(대칭성).
z 점수 / 순위 정규화	단위가 다른 값을 비교 가능하게 만드는 방법. z 는 (값-평균) ÷ 표준편차, 순위 정규화는 순위를 정규분포 점수로 바꾼다. 후자는 분포 모양에 안 휘돌린다.
AUC	어떤 점수가 두 부류(패스를 받은 선수 / 안 받은 선수)를 얼마나 잘 가르는지를 0.5~1 로 나타낸 값. 0.5는 동전 던지기와 같고 1은 완벽하다.
순열검정	정답을 무작위로 섞은 가짜 데이터에서도 같은 성능이 나오는지 확인하는 검정. "우연히 좋아 보이는 것"을 걸러낸다.
모션블러	빠르게 움직이는 물체가 한 프레임 안에서 번져 찍히는 현상. 공 검출을 어렵게 만드는 주범.
usable	기여도 계산에 실제로 쓰인 프레임. 소유팀이 정해졌고 양 팀 인원이 충분히 추적된 프레임만 해당된다.

부록 A

tracks.csv 열 전체 표

열	단위/범위	뜻	기여도 계산에서의 쓰임
frame	정수, 0부터	원본 영상의 프레임 번호(구간 분석이어도 원본 기준)	필수 — 시간축의 기준
time_sec	초	영상 시작 기준 경과 시간	fps 추정, 표본검사 시각 표시
track_id	정수, 놓치면 -1 공은 1,000,000~	같은 선수는 같은 번호	필수 — 선수별 집계 기준. 공 행 판별에도 쓰인다
class_id / class_name	—	player / goalkeeper / referee / ball / person	역할 판별용(현재 열 이름이 안 맞아 인식 안 됨 → 6.2)
conf	0~1	검출 신뢰도	공 신뢰도 하한 필터에 쓸 수 있다
x1,y1,x2,y2	픽셀	박스 좌상단·우하단	직접 안 씀 (draw_boxes 용)
cx, cy	픽셀	박스 중심	공 위치로 쓰인다(공은 지면에 안 붙어 있으므로)
fx, fy	픽셀	발밀 중심점 = $((x1+x2)/2, y2)$	가장 중요 — 호모그래피를 적용해 X, Y(미터)로 만들 좌표
w, h	픽셀	박스 너비·높이	공 크기 검사의 기준자(h 의 중앙값)
team	0 / 1 / -1 / 빈칸	양 팀 / 팀 아님 / 공	필수 — 공격팀·수비팀을 가른다 (규약 변환 필요 → 6.3)
interpolated	0 / 1	1이면 측정이 아니라 메운 추정 위치	현재 무시됨 → 6.5

부록 B

주요 상수 표

tracker.py — 측정 단계

상수	기본값	뜻과 근거
match_thresh	0.9	트랙 연결 허용 상한. 0.8→0.9 로 ID 233→139개
track_buffer	30	가려진 선수 기억 프레임 수
static_max_move	0.10	(이동폭/키)가 이보다 작으면 중계 그래픽
ball_ratio_min / max	0.16 / 0.36	공 크기 ÷ 사람 키 중앙값. 진짜 공 0.170~0.319
ball_min_bright	215	흰 덩어리 최대 밝기. 공 248 · 마크 196
ball_min_fill	0.65	채움 비율. 공 0.77(원은 0.785) · 축구화 0.52
ball_reach	140 px	공 후보 연속성 점수가 절반이 되는 거리
ball_switch_margin	0.20	이만큼 확신하는 후보가 있으면 갈아탄다
interp_max_gap_sec	0.4초	공 보간 상한. 0.8초면 오차가 선수 키의 0.28
keeper_track_conf	0.50	트랙 신뢰도 중앙값. 골대 0.22~0.41 · 골키퍼 0.60~0.81
team_merge_ratio	0.5	남은 색 군집을 팀으로 흡수하는 거리 기준
ball_id_base	1,000,000	공 전용 모델의 track_id 시작 번호

soccer_dpi_pa_v7.py — 해석 단계

상수	기본값	뜻과 근거
GRID_STEP	1.0 m	SC 격자 크기. 0.5로 하면 90분에 2시간 이상 걸린다
LAMBDA_GOAL / BALL	18 / 20 m	위험도 감쇠 거리(골대 / 공)
PR_R	10 m	압박권 반경. 이 밖에서는 PR = 0
PR_VMAX	5 m/s	접근속도 정규화 기준
PR_WD : PR_WV	0.6 : 0.4	PR 내부 가중치(거리 : 속도). 검증 전 가설
W_LANE	3 m	패스선이 완전히 열렸다고 볼 거리. 15m 패스가 1.0~1.2초 걸리고 그 사이 수비수가 2m 이동 + 팔다리 1m
PA_SPACE_R	8 m	수신 공간 만점 거리
PA_FWD_REF	30 m	전진 이득 정규화 기준
PA_MIN_PASSES	200	지수 학습에 필요한 최소 패스 수
SPEED_MAX	11 m/s	= 39.6 km/h. 프로 실측 최고 34~37 km/h
POSS_RADIUS	자동 2~8 m	공 소유 인정 거리
HOLD_FRAMES	1.5초	소유팀 전환 인정에 필요한 유지 시간
GK_MAX_DIST	22 m	자기 골대까지 평균거리. 골키퍼 ~20m · 센터백 ~31m
W_SC : W_PR	0.5 : 0.5	DPI 가중치. 선형적 근거가 없어 동일하게 둔다
TC 가중치	1/3 씩	순위에 맞춰 조정하면 결론을 먼저 정하는 것이 된다
TC_NORM	rank	순위 정규화. PR 의 긴 꼬리가 합계를 끌고 가는 것을 막는다

이 문서의 모든 수치는 두 가지 출처 중 하나다 — (1) 코드 주석에 기록된 실측값, (2) 이 문서를 쓰면서 코드를 직접 실행해 확인한 값 (6.2 · 6.4 · 7장의 재현 실험). 추측으로 채운 숫자는 없다.